



Algoritmos e Estruturas de Dados II

Trabalho 1

Desafio na cozinha

Erick Jacquin precisa de ajuda. A falta de padrão nos cardápios, o acesso lento às informações das receitas e a má organização dos ingredientes estão deixando o chef irritado. Em uma cozinha profissional, encontrar rapidamente uma receita, saber quais ingredientes estão disponíveis, organizar menus por custo, tempo de preparo e dificuldade, além de recomendar pratos adequados para cada situação, é essencial para o bom funcionamento do restaurante.

Além da desorganização no sistema de receitas, um novo problema surgiu: receitas estão sendo adulteradas misteriosamente. Ingredientes somem, tempos de preparo mudam e pratos clássicos aparecem com versões suspeitas.

| *“Isso aqui virou sabotagem culinária!”* — provavelmente diria Jacquin.

Ou seja, não basta apenas organizar o restaurante. É preciso garantir também a integridade das receitas.

Diante desse problema, sua equipe, composta por até dois integrantes, foi contratada para desenvolver o Desafio na Cozinha, um sistema inteligente de gerenciamento de receitas e menus. O objetivo é organizar o acervo culinário do restaurante, permitindo buscas eficientes, recomendações automáticas e otimização na escolha dos pratos, além de garantir que as receitas não foram alteradas indevidamente.

Nesse trabalho, você e sua dupla, se houver, deverão implementar um sistema que utilize diferentes estruturas de dados, como Tries, Árvores B e Tabelas Hash, além de aplicar pelo menos um algoritmo guloso para resolver um problema de recomendação, busca ou otimização.

Para evitar sabotagens culinárias, o sistema deverá manter um identificador baseado no conteúdo de cada receita, que permita verificar se uma receita foi modificada e detectar inconsistências no sistema, como: receitas duplicadas com conteúdo diferente, alterações não autorizadas, inconsistência entre os dados armazenados e os resultados retornados pelas consultas.

O trabalho poderá ser desenvolvido em uma linguagem de programação de preferência do aluno, desde que a escolha permita a implementação eficiente das funcionalidades exigidas. O código entregue deverá estar claro, funcional, organizado e devidamente documentado.

➤ Contexto

O restaurante possui diversas receitas cadastradas, cada uma com informações como nome, categoria, ingredientes, tempo de preparo, custo estimado, nível de dificuldade, avaliação dos clientes e quantidade de pedidos.

▢ Fonte de dados

O grupo (dupla) deverá escolher uma fonte de dados para popular o sistema. A fonte poderá ser uma API pública, como a [TheMealDB](#) ou [RecipeBox](#), ou um dataset estático, como bases de receitas em arquivos .csv ou .json, por exemplo o [Recipe Ingredients Dataset](#).

A base escolhida deverá conter, preferencialmente, no mínimo 50 receitas. Cada receita deverá possuir informações como nome, categoria, ingredientes e algum critério numérico que possa ser usado para ordenação ou recomendação, como tempo de preparo, custo, avaliação ou popularidade.

A fonte de dados utilizada deverá ser informada no README, explicando quais dados foram coletados, como foram carregados e quais adaptações foram realizadas.

➤ Módulos do sistema:

Módulo 1 — Livro de Receitas: responsável por carregar, armazenar e listar as receitas disponíveis no sistema.

Módulo 2 — Busca Rápida no Cardápio: responsável por permitir buscas eficientes por nome, ID, categoria ou prefixo de receita.

Módulo 3 — Organização dos Ingredientes: responsável por relacionar os ingredientes às receitas cadastradas e permitir consultas por ingrediente.

Módulo 5 (Desafio)— Recomendação do Chef: responsável por sugerir receitas ou menus com base em critérios como custo, tempo de preparo, avaliação, popularidade ou ingredientes disponíveis.

➤ Modos de Interação:

Os modos de interação representam formas de uso do sistema e podem reutilizar funcionalidades implementadas nos módulos descritos anteriormente.

-  Modo Investigação: encontrar receitas corrompidas
 - O sistema deve permitir identificar possíveis inconsistências nas receitas armazenadas.

Nesse modo, o usuário deverá ser capaz de:

- Verificar se uma receita foi alterada desde sua inserção no sistema
- Identificar receitas com conteúdos inconsistentes ou duplicados
- Detectar possíveis conflitos entre versões de uma mesma receita
- Validar a integridade dos dados carregados a partir de arquivos ou APIs

As verificações devem ser eficientes, mesmo para grandes volumes de dados.

- 🍳 Modo Chef: recomendar pratos sob restrições

- O sistema deve auxiliar na escolha de receitas ou na composição de menus com base em critérios definidos.

O usuário deverá ser capaz de:

- Selecionar receitas com base em restrições (ex: tempo máximo, orçamento, dificuldade)
- Obter sugestões de pratos considerando múltiplos critérios
- Priorizar receitas com base em métricas como avaliação ou popularidade
- Gerar combinações de receitas que atendam a um objetivo específico (ex: menu econômico, menu rápido, etc.)
- As decisões devem ser justificáveis com base nos critérios definidos.

- 🔍 Modo Consulta Rápida

- O sistema deve permitir a recuperação eficiente de receitas com base em diferentes critérios.

O usuário deverá ser capaz de:

- Buscar receitas por nome (total ou parcial)
- Filtrar receitas por categoria
- Consultar receitas a partir de ingredientes específicos
- Localizar rapidamente receitas com base em identificadores únicos

- O foco deste modo é garantir desempenho e eficiência na recuperação das informações.

➤ **Requisitos Obrigatórios :**

O projeto deverá implementar do zero e utilizar, obrigatoriamente, pelo menos 3 das 4 seguintes técnicas: Tabelas Hash; Árvore Trie (ou Árvore Patrícia); Árvore B (ou B+); Algoritmo Guloso.

As estruturas de dados centrais do sistema deverão ser implementadas pelo grupo, de forma a demonstrar compreensão de seu funcionamento.

O uso de bibliotecas prontas é permitido como apoio, desde que não substitua integralmente as estruturas principais exigidas no trabalho.

Caso bibliotecas sejam utilizadas, o grupo deverá justificar sua escolha e explicar claramente seu funcionamento e impacto na solução.

Bibliotecas também podem ser utilizadas para: leitura de arquivos; consumo de API; manipulação de JSON ou CSV; interface; exibição de dados.

▣ **Entregáveis:**

O grupo deverá entregar:

1. Repositório do Projeto: O link do repositório deve estar público (ou acessível ao professor) e ser informado claramente no README.
2. Entrega no e-Aula (Oficial): O grupo deverá baixar o repositório e enviar no e-Aula um arquivo .zip contendo o último commit funcional realizado dentro do prazo. Apenas envios pelo e-Aula serão avaliados.
3. Arquivo README (Relatório Final): Deve estar na raiz do projeto contendo: o link do repositório, instruções claras de execução, a fonte de dados escolhida, explicação de quais estruturas (das 4 opções) foram implementadas e onde foram aplicadas, e justificativas para os algoritmos escolhidos.

4. Arquivo de Dados: O arquivo CSV/JSON utilizado (caso não seja uma API externa), para que o professor consiga rodar o código localmente

Obs: Todos os integrantes deverão demonstrar domínio sobre o código e sobre as decisões de implementação. Caso o sistema utilize uma API, recomenda-se salvar os dados localmente para evitar problemas de conexão no dia da apresentação.

Critérios de avaliação:

A nota final do trabalho será calculada como a média harmônica entre os componentes de avaliação: Código (80%) e Apresentação (20%).

$$M_h = \frac{w_1 + w_2}{\frac{w_1}{n_1} + \frac{w_2}{n_2}}$$

Onde:

- w_1 e w_2 são os pesos das partes do trabalho (código = 80 e apresentação = 20)
- n_1 é a nota obtida no Código.
- n_2 é a nota obtida na Apresentação.
- Código (80%)
 - O código implementado será avaliado com base nos seguintes critérios:
 - Corretude: Funcionamento correto e sem erros durante testes.
 - Completude: Implementação de todas as funcionalidades exigidas no enunciado.
 - Organização e Legibilidade: Estrutura clara, boas práticas de programação e documentação adequada.

- O uso inadequado de bibliotecas que substituam os objetivos do trabalho poderá impactar negativamente a nota.
- Entregáveis:
 - README: Deve conter instruções de execução, descrição das funcionalidades e critérios de decisão.
 - Código Implementado: Arquivos contendo a solução completa e funcional.
- Apresentação (20%):
 - A apresentação será avaliada de forma individual e deverá ocorrer em sala de aula na data definida pelo docente. A nota individual considerará:
 - Domínio do Conteúdo: Clareza e segurança ao explicar o código implementado.
 - Capacidade de Resposta: Habilidade em responder a perguntas e realizar alterações solicitadas no código para validação do conhecimento.
 - Engajamento: Participação ativa durante a apresentação.
 - Formato de apresentação: Caso o trabalho seja realizado em dupla, todos os integrantes devem demonstrar domínio completo do código e das decisões tomadas no desenvolvimento. A distribuição das partes da apresentação será definida pelo docente no momento da avaliação.
 - Observação: caso o aluno não demonstre conhecimento sobre o código entregue para avaliação, a nota geral do trabalho tenderá a zero.
- Plágio: Trabalhos com similaridade igual ou superior a 70% a outros receberão nota 0 ;)

Entrega: até 18 de maio de 2026 às 12h (meio dia).

Apresentação: 18 de maio de 2026.